Verification Plan for 8-bit ALU

ALU diagram and specification

- 8-bit Arithmetic Logic Unit (ALU).
- Purely combinational module driven by 3-bit opcode.
- Result output selected according to 3-bit opcode (8 distinct operations).
- Result computed from 8-bit input A and 8-bit input B.
- Operations supported: Pass-through A (000), Addition A+B (001), Subtraction A-B (010), Increment A+1 (011), Decrement A-1 (100), Bitwise AND A&B (101), Bitwise OR A|B (110), Bitwise NOT ~A (111). Default operation outputs A.
- carryout flag active on addition carry generation or subtraction borrow/carry generation.
- carryout flag is explicitly set to 0 as default before opcode evaluation (forces carryout = 0 for non-ADD/SUB operations).
- zero_flag active high whenever Result value is 0.

Signals

1. A -> 8-bit input operand A; extreme boundary values (8'h00, 8'hFF, walking 1s/0s).

2. B -> 8-bit input operand B; extreme boundary values (8'h00, 8'hFF, walking 1s/0s).

3. opcode -> 3-bit control input selection signal; 8 functional operations (3'b000 to 3'b111).

4. Result -> Checker:

- If opcode == 3'b000 -> check Result == A
- If opcode == 3'b001 -> check {carryout, Result} == A + B
- If opcode == 3'b010 -> check {carryout, Result} == A - B
- If opcode == 3'b011 -> check Result == A + 1
- If opcode == 3'b100 -> check Result == A - 1
- If opcode == 3'b101 -> check Result == A & B
- If opcode == 3'b110 -> check Result == A | B
- If opcode == 3'b111 -> check Result == ~A
- If default opcode -> check Result == A

5. carryout -> Checker:

- If opcode == 3'b001 -> check carryout == (A + B) >> 8
- If opcode == 3'b010 -> check carryout == (A - B) >> 8
- If opcode in {3'b000, 3'b011, 3'b100, 3'b101, 3'b110, 3'b111, default} -> check carryout == 1'b0 (verify immediate zero clear)

6. zero_flag -> Checker: if Result == 0 -> check zero_flag == 1, else == 0.

Functional coverage

We need to cover all output signals, operation combinations, and corner conditions:

- Result -> cover all values (0 to 255) and extreme boundary values (8'h00, 8'hFF).
- Result -> toggling only (bit-level toggle 0 -> 1 and 1 -> 0 for all 8 bits).

- Result/ADD -> out overflow from 255 to 0.
- Result/SUB -> out underflow from 0 to 255.
- Result/INC -> out overflow from 255 to 0.
- Result/DEC -> out underflow from 0 to 255.
- zero_flag -> 1 iff (Result == 0).
- zero_flag -> 0 iff (Result != 0).
- carryout -> 1 iff carry generated in ADD / SUB.
- carryout -> 0 iff no carry generated in ADD / SUB.
- carryout default clearing -> verify carryout == 0 for all non-ADD/SUB opcodes immediately after an overflow condition.
- opcode -> cover all 8 opcode selection values (3'b000 to 3'b111).
- opcode x zero_flag -> cross coverage of opcode and zero_flag state.
- opcode x carryout -> cross coverage of opcode and carryout flag state.
- Operands A, B -> cover boundary patterns (8'h00, 8'hFF, walking 1s, walking 0s, alternating bits).

Sequences

Sequences & Constrained Randomization

1. Opcode sweep sequence -> sweep opcode sequentially from 3'b000 to 3'b111 with fixed non-zero operands.

2. Boundary sequence -> test combinations of boundary values for A and B (8'h00, 8'h01, 8'h7F, 8'h80, 8'hFE, 8'hFF).

3. Carry sequence -> constrain inputs to trigger carryout == 1 for ADD (3'b001) and SUB (3'b010).

4. Zero flag sequence -> constrain inputs to produce Result == 0 across all 8 opcodes.

5. Carryout clearing sequence -> alternate between ADD/SUB with carryout = 1 and non-ADD/SUB opcodes to verify carryout resets to 0 immediately.

6. Random sequence -> fully randomized inputs A, B, and opcode for at least 1000 transactions.

Test scenarios

- **Scenario 1: Opcode sweep sequence -> random operand transaction range [0:255].**
  - Goal: testing basic execution of all 8 ALU operations (Pass-through, ADD, SUB, INC, DEC, AND, OR, NOT).
- **Scenario 2: Boundary sequence -> testing boundary values and overflow/underflow.**
  - Goal: testing arithmetic overflow wraparound (255 + 1 -> 0 for ADD/INC) and underflow (0 - 1 -> 255 for SUB/DEC).
- **Scenario 3: Carryout generation and zero-clear sequence -> switching between arithmetic and logical opcodes.**
  - Goal: verifying carryout generation in ADD/SUB and ensuring carryout immediately resets to 0 for non-arithmetic opcodes.
- **Scenario 4: Zero flag sequence -> opcode sweep targeting zero result condition.**
  - Goal: testing zero_flag generation active high whenever Result == 0 for all applicable opcodes.
- **Scenario 5: Random sequence -> randomized operands A, B, and opcode for at least 1000 times.**
  - Goal: testing dynamic combinational response, random coverage, and continuous scoreboard verification.